低レベルは良いぞと言う話

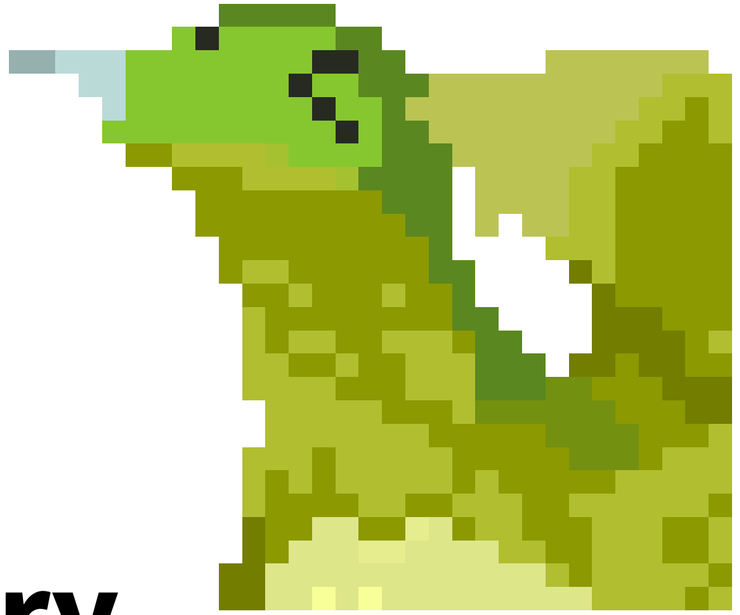
URL: https://canva.link/ycqbquww3sjtz9p

ぷろふぃーる

HN: 緑

学部: 2年

何してる: ゲーム開発



綺麗なTwitter : @Tose_Midry

GitHub : midry3

ホームページ : tmidry.com

任天堂の求人を見たことはありますか？

理工系の仕事

ゲーム開発

ゲーム開発とは？

ゲーム開発では、任天堂のゲーム専用機に向けたゲームソフトの開発と、その品質を保証するための検証工程（QA=Quality Assurance）、そして、ゲーム体験の入り口となるゲーム専用機の本体機能の開発を担当します。ゲーム開発の仕事は、ゲームソフトや本体機能などのアプリケーションだけを作るのではなく、ゲームエンジンなども含めて、良いものを作るために技術力を発揮して幅広く開発を行います。さまざまな領域のエンジニアが、プランナーやデザイナー、サウンドスタッフなどのチームメンバーと議論を繰り返しながら、お客様にお届けする商品やサービスを作りこむ仕事です。

ゲームソフトの開発

ゲームソフトの開発で、エンジニアが担う開発対象は多岐にわたります。たとえばグラフィックスや物理演算、キャラクター制御、サウンド、UI、ゲームシステム、通信といった、ゲーム専用機の中で動くプログラムのほか、PCツールやWebアプリ、ゲームサーバーなど、ゲーム専用機の外で動くプログラムまで、さまざまなプログラムを作成します。また、それらのプログラムは、ひとつのゲームタイトルに特化したものから、複数のタイトル間で共有できるよう汎用化されたもの（=ゲームエンジン）まで、形態もさまざまあり、これらもすべて、エンジニアが作り上げています。

Nintendo

キャリア採用

募集要項

京 都 コンパイラー・ツールチェイン開発エンジニア

業務内容	任天堂ゲームプラットフォームは独自のOSの上に構築されています。 このOS上で動くプログラムのためのコンパイラー・ツールチェインや、ランタイムライブラリの開発を行います。 ※ランタイムライブラリとは、C/C++の標準ライブラリやプログラムのスタートアップルーチンなどの、プログラミング言語の機能を提供するために必要なライブラリを指します。
適性	【必須の実務経験と能力】 ● C/C++言語による開発経験 ● 以下のいずれかの知識・経験 -コンパイラーに関する知識・開発経験 -ランタイムライブラリの実装に関連する知識・開発経験 ※ランタイムライブラリの実装に関連する開発経験とは、例えば以下のようなものを指します。 - 言語処理系のランタイムライブラリ・標準ライブラリの実装 - ベアメタルプログラミング - 標準ライブラリを利用しないプログラミング - システムコールを直接使用するプログラミング 【望ましい実務経験と能力】 - 英語によるコミュニケーション能力 - セキュリティやコンパイラー技術に関する最新の動向に追従していること - 独自OS向けの言語処理系開発に対するモチベーション 以下のような知識や経験をお持ちの方を歓迎します。

サイゲの求人を見たことはありますか？

コンシューマーエンジニア ゲーム開発部門

コンシューマーゲームのコンテンツやライブラリの設計、開発やテスト、運用を行います。C++、C#、Python、Lua、Goなどを用い、プロジェクトにより内製エンジン・外部エンジンを使い分けています。中規模～ハイエンドタイトルやネットワークゲームに至るまで幅広いゲーム開発を行っています。

先輩スタッフのインタビューを見る

ゲームエンジンエンジニア ゲーム開発部門

コンシューマーゲーム向けの内製ゲームエンジン「Cyllista Game Engine」の開発を行います。
主に C++ と Python を使用します。
グラフィックス・アニメーションなどゲームを支える様々な技術の開発や、ゲームを作るための環境・ツールの開発を行います。
ゲーム開発者が「最高のコンテンツ」を制作する基盤を提供しています。

大きい企業ほどUnityやUEなどは使っておらず、エンジンを自作している



実は低レベル技術者の方が需要が高い

よってC++をやろう！

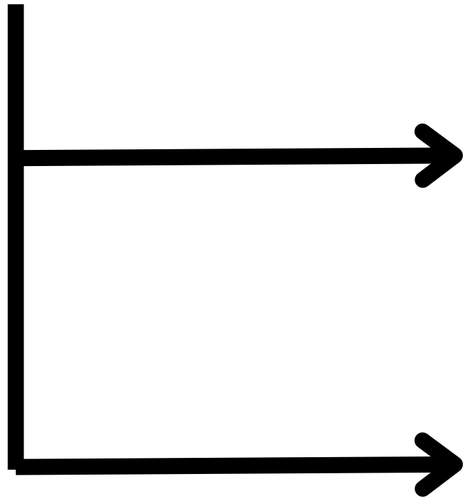
ところでゲーム開発用のフレームワークを
自作しているので、今回はその話

言語: C++

バックエンド: Dxライブラリ

シーン構造

Parent (position, scale, rotation, etc)



Child1

Child2

親が動けば子も連動する

シーン構造

Parent.position.x += 1;

Child1もx方向に+1
Child2もx方向に+1

シーン構造

Child1.position.x += 1;

動くのはChild1だけ

主要メソッド定義

virtual void _Load() {};
初期化処理

virtual void _Release() {};
解放処理

virtual void _Update(float dt) {};
更新処理

virtual void _Draw() const {};
描画処理

virtual(仮想関数)にすると、
アップキャストしても
override先が呼ばれる！

補足: アップキャスト

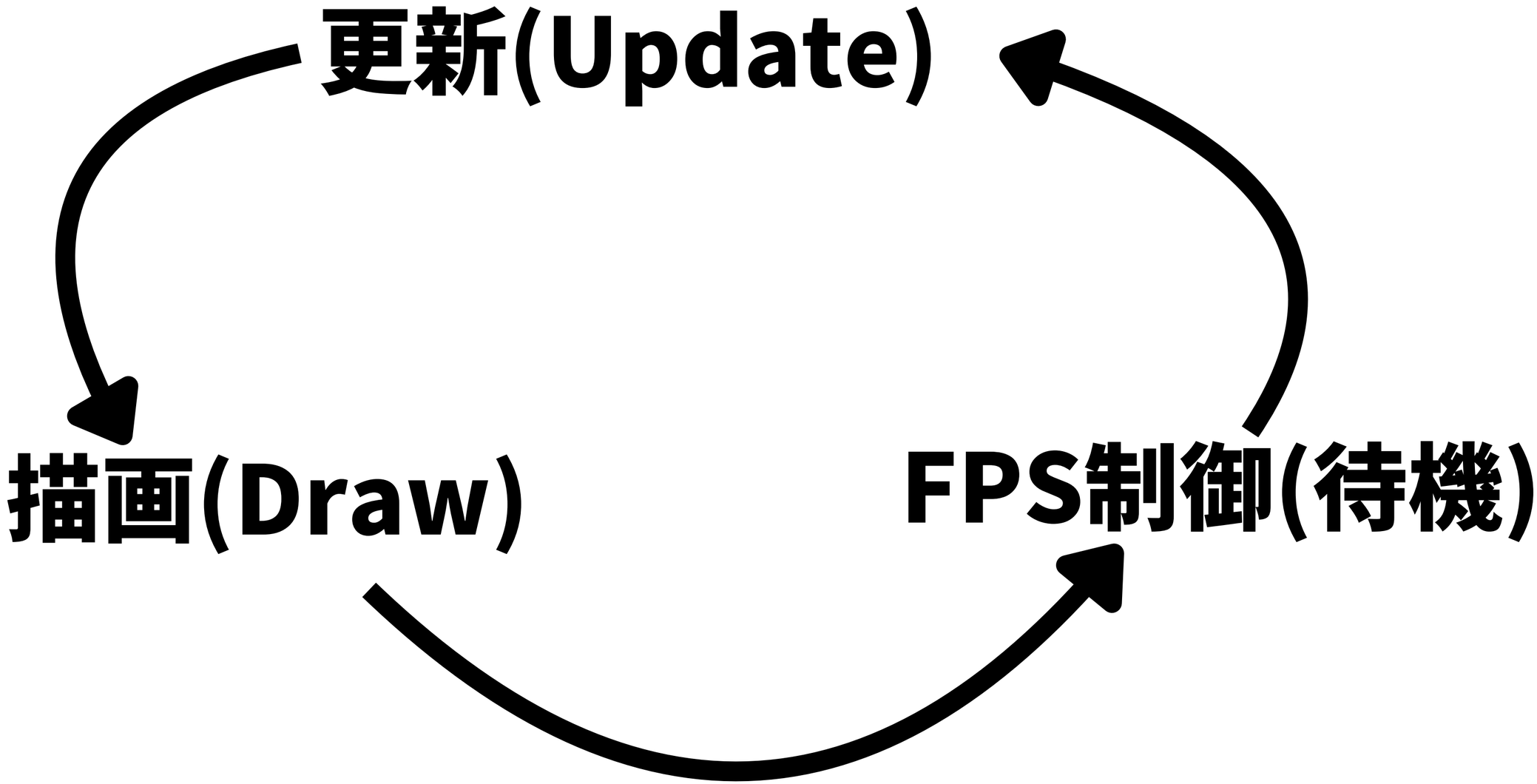
class Scene {};

class TextLabel : public Scene {};

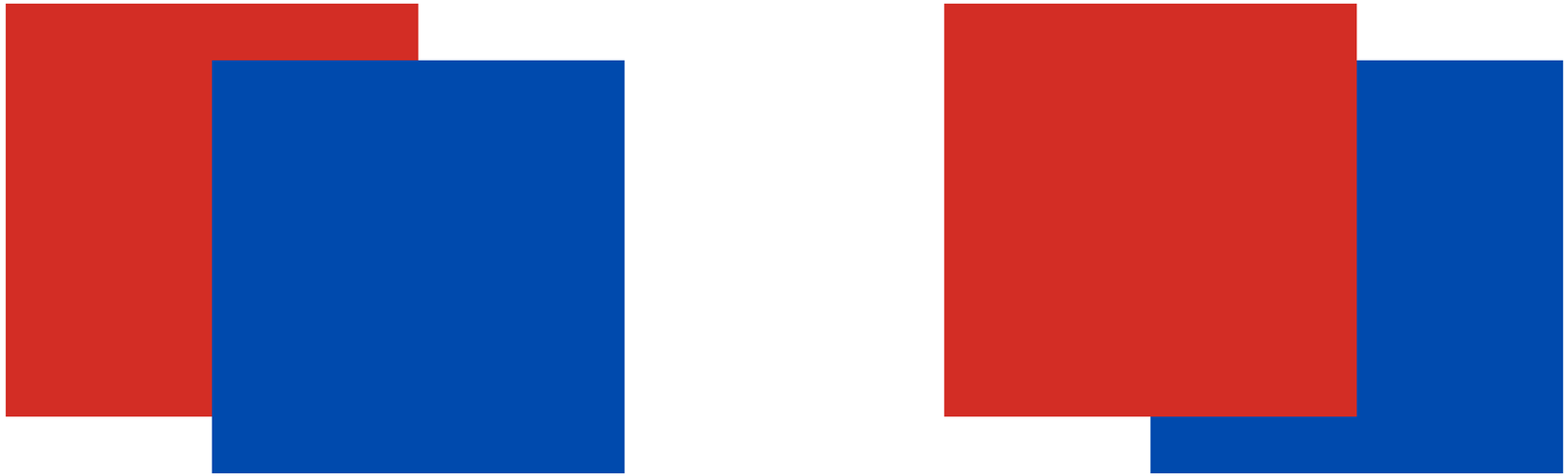
```
TextLabel label = TextLabel();
Scene labelScene = label;
```

基底クラスへのキャストのこと

メインループ



描画順の管理



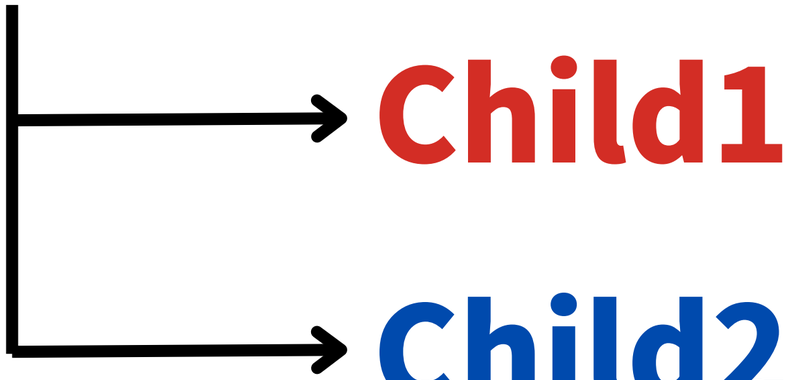
描画順は大事

描画順の管理

先



後



基本はシーンの追加順

しかし優先度を明示したい場合…？

描画順の管理

z-indexに対応する



Child1(z=1)

Child2(z=0)

(2次元平面においてz座標は奥行きを表す)

描画順の管理: どっちが高速？

1. ソートしながらinsert

A0(z=0) A1(z=0) B0(z=2) B1(z=2)



ここやね

C0(z=1)

2. 最後にソート

A0(z=0) B0(z=2) A1(z=0) B1(z=2)

描画順の管理: どっちが高速？

1. ソートしながらinsert

O(N^2)

2. 最後にソート

O(N logN)



脱線: Yソート



脱線: Yソート

- Y座標でソートする





脱線: Yソート

- z-indexと併用する

まずz-indexでソートし、同じz-index内でyソートする

当たり判定の話

参考

https://canva.link/fam2o4ufecf4p1k

アニメーション処理

参考

https://canva.link/4c1h7bychqwi9gg

API設計のコツ

中のコードはどれだけ汚くても、呼び出し関数の引数だけは簡潔に！！

```
CreateTimer(0.1, [this]() {
    Jump();
    });
```

```
CreateTweenGroup()->CreateTween(&alpha, 255, 0.5)->From(0);
```

メソッドチェーン

```
_sprite->AddAnimation(PlayerAnimation::Idle, LoadAsset<SpriteSheet>("character/plea/idle.png"))
    ->AddAnimation(PlayerAnimation::Ride, LoadAsset<SpriteSheet>("character/plea/ride.png"))
    ->AddAnimation(PlayerAnimation::Fly, LoadAsset<SpriteSheet>("character/plea/flying.png"))
    ->Play(PlayerAnimation::Idle);
```

```
_sprite->AddAnimation(PlayerAnimation::Idle, LoadAsset<SpriteSheet>("character/plea/idle.png"));
_sprite->AddAnimation(PlayerAnimation::Ride, LoadAsset<SpriteSheet>("character/plea/ride.png"));
_sprite->AddAnimation(PlayerAnimation::Fly, LoadAsset<SpriteSheet>("character/plea/flying.png"));
_sprite->Play(PlayerAnimation::Idle);
```

メソッドチェーン

```
_sprite->AddAnimation(PlayerAnimation::Idle, LoadAsset<SpriteSheet>("character/plea/idle.png"))
    ->AddAnimation(PlayerAnimation::Ride, LoadAsset<SpriteSheet>("character/plea/ride.png"))
    ->AddAnimation(PlayerAnimation::Fly, LoadAsset<SpriteSheet>("character/plea/flying.png"))
    ->Play(PlayerAnimation::Idle);
```

メソッドにてreturn this; をすることにより、メソッドを連続して呼べる

メソッドチェーン

```
        if (loop) {
            spriteSheet->SetLoop(true);
        }
        SetScale(spriteSheet->GetScale());
        _spriteSheetManager.Play();
        AnimationChanged(_currentAnimation);
        return this;
    }
    AnimatedSprite2D* Play(const String& anim, bool loop = false) {
        auto it = std::find(_animationNames.begin(), _animationNames.end(), anim);
        if (it != _animationNames.end()) {
            return Play(std::distance(_animationNames.begin(), it), loop);
        }
        return this;
    }
    AnimatedSprite2D* AddAnimation(const int index, const SharedPtr<SpriteSheet>& spriteSheet) {
        _spriteSheetManager.RegisterSpriteSheet(index, spriteSheet);
        _animationNames.push_back(std::to_string(index));
        return this;
    }
    AnimatedSprite2D* AddAnimation(const String& anim, const SharedPtr<SpriteSheet>& spriteSheet) {
        _spriteSheetManager.RegisterSpriteSheet(_spriteSheetManager.GetAnimationCount(), spriteSheet);
        _animationNames.push_back(anim);
        return this;
    }
    [[nodiscard]] Vector2D GetCurrentSheetSize() const {
        return _spriteSheetManager.GetCurrentSheetSize();
    }
    [[nodiscard]] Vector2D GetCurrentFrameSize() const {
        return _spriteSheetManager.GetCurrentFrameSize();
    }
```

シェーダー

シェーダー

- 頂点シェーダー
- ピクセルシェーダー

シェーダー

```
#pragma once

Texture2D Texture0 : register(t0);
SamplerState Sampler0 : register(s0);

struct PS_INPUT
{
    float4 Position : SV_POSITION;
    float4 Diffuse : COLOR0;
    float4 Specular : COLOR1;
    float2 UV : TEXCOORD0;
    float2 SubUV : TEXCOORD1;
};
```

レジスタtの0番目

レジスタsの0番目

構造体定義

シェーダー

```hlsl
#include "PixelShaderBase.hlsli"

cbuffer OutLineShaderParam : register(b4)
{
    float4 GlowColor;    // 色
    float OutlineWidth; // 輪郭（px）
    float Time;          // 時間経過（sec）
    float2 TextureSize; // テクスチャの解像度(w, h)
};

float4 main(PS_INPUT input) : SV_TARGET
{
    float4 color = Texture0.Sample(Sampler0, input.UV);
    float2 texelSize = (1.0f / TextureSize) * OutlineWidth;
    float alpha = -4.0 * color.a;

    alpha += Texture0.Sample(Sampler0, input.UV + float2(texelSize.
    alpha += Texture0.Sample(Sampler0, input.UV + float2(-texelSize
    alpha += Texture0.Sample(Sampler0, input.UV + float2(0.0, texe
```

シェーダー：定数バッファ

- C++側からパラメーター調整をするためのデータ

```
struct alignas(16) OutLineShaderParam {
    float glowColor[4];    // RGBA
    float outlineWidth;    // 輪郭の太さ
    float time;            // 時間
    float textureSize[2]; // 幅, 高さ
};
```

シェーダー：定数バッファ

レジスタbの4番目



```
cbuffer OutLineShaderParam : register(b4)
{
    float4 GlowColor;    // 色
    float OutlineWidth; // 輪郭（px）
    float Time;          // 時間経過（sec）
    float2 TextureSize; // テクスチャの解像度(w, h)
};
```

シェーダー：定数バッファ

ハマりやすいポイント

構造体のサイズの合計を16の倍数にする！！！

```
cbuffer OutLineShaderParam : register(b4)
{
    float4 GlowColor;    // 色
    float OutlineWidth; // 輪郭（px）
    float Time;             // 時間経過（sec）
    float2 TextureSize; // テクスチャの解像度(w, h)
};
```

float: 4bytes

シェーダー：定数バッファ

ハマりやすいポイント

パラメーターの並びを一致させる！！！

```
cbuffer OutLineShaderParam : register(b4)
{
    float4 GlowColor;   // 色
    float OutlineWidth; // 輪郭（px）
    float Time;         // 時間経過（sec）
    float2 TextureSize; // テクスチャの解像度(w, h)
};
```

```
struct alignas(16) OutLineShaderParam {
    float glowColor[4];   // RGBA
    float outlineWidth;   // 輪郭の太さ
    float time;           // 時間
    float textureSize[2]; // 幅，高さ
};
```

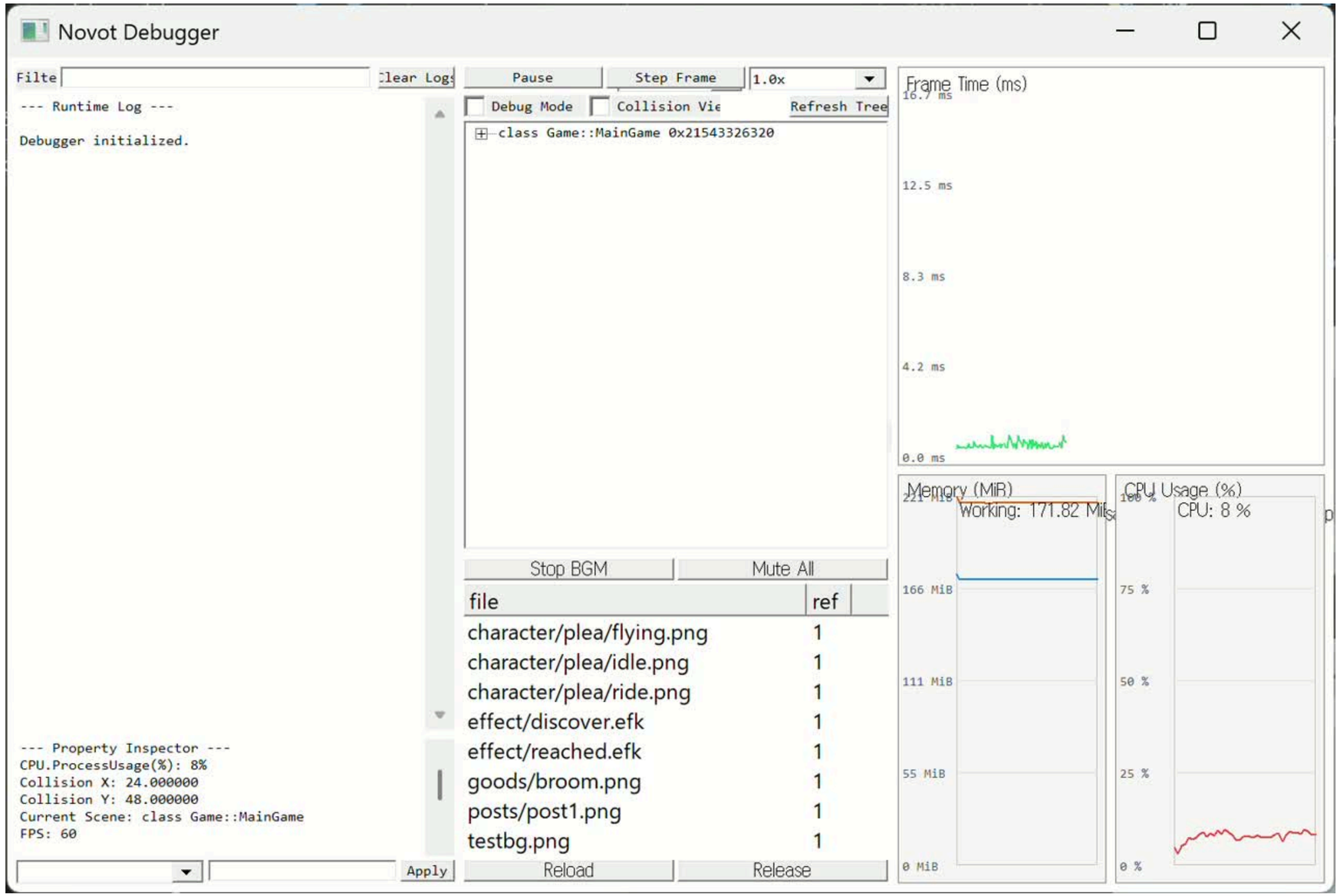
シェーダー：定数バッファ

なぜこんな制約が？

=> GPUがそういう仕様だから

デバッガーの自作！！！(たのしい)

デバッガーの自作

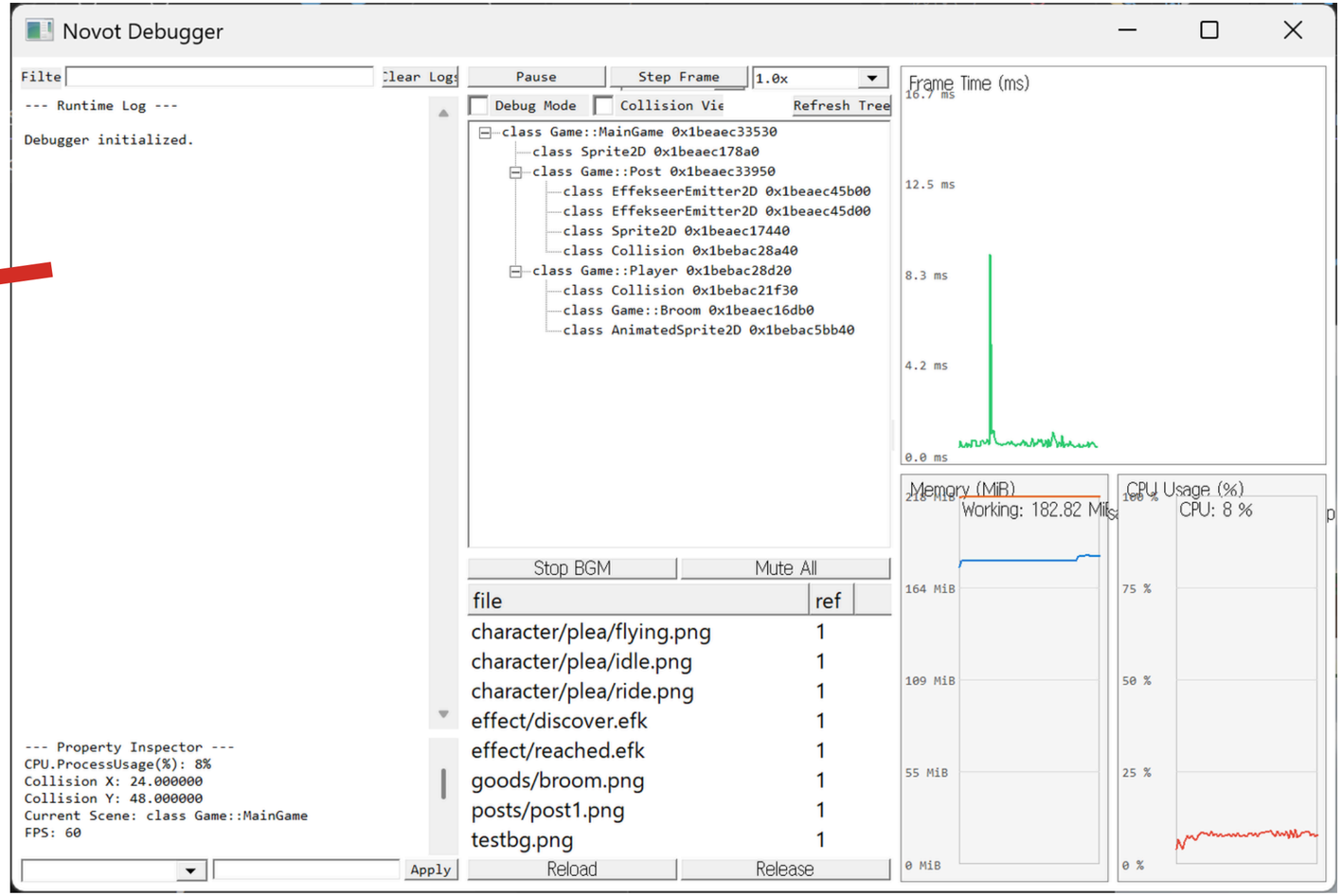

Novot Debugger
Filte
lear Logs
--- Runtime Log ---
Debugger initialized.
Pause
Step Frame
1.0x
Debug Mode
Collision Vie
Refresh Tree
class Game::MainGame 0x21543326320
Frame Time (ms)
16.7 ms
12.5 ms
8.3 ms
4.2 ms
0.0 ms
Memory (MiB)
221 MiB
Working: 171.82 MiB
166 MiB
111 MiB
55 MiB
0 MiB
CPU Usage (%)
100 %
CPU: 8 %
75 %
50 %
25 %
0 %
Stop BGM
Mute All
file
ref
character/plea/flying.png 1
character/plea/idle.png 1
character/plea/ride.png 1
effect/discover.efk 1
effect/reached.efk 1
goods/broom.png 1
posts/post1.png 1
testbg.png 1
--- Property Inspector ---
CPU.ProcessUsage(%): 8%
Collision X: 24.000000
Collision Y: 48.000000
Current Scene: class Game::MainGame
FPS: 60
Apply
Reload
Release

デバッガーの自作

- 単純に画面がかっこよくて「良い」

デバッガーの自作

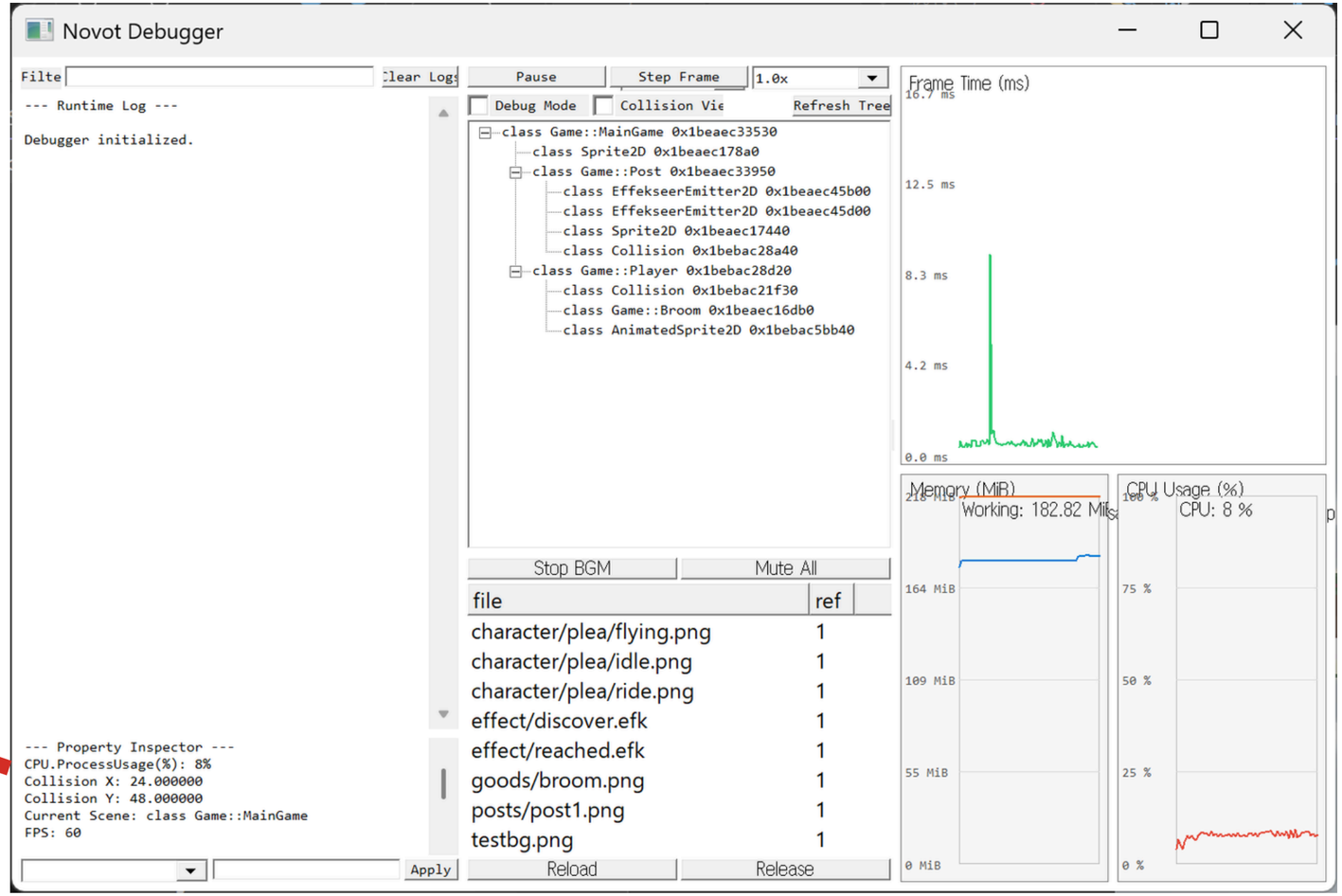
ログ



みんな大好きPrintデバッグで使う

デバッガーの自作

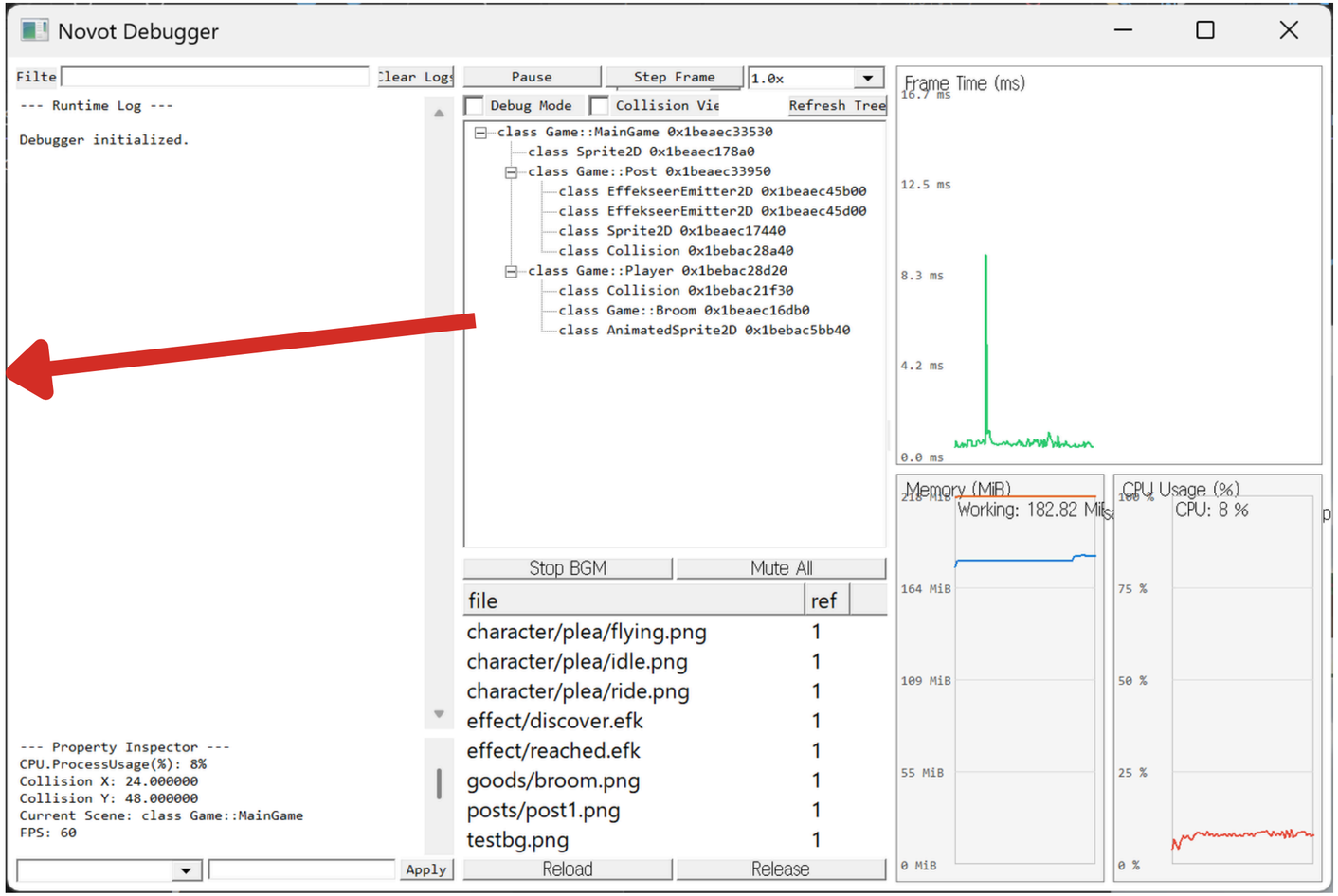
プロパティ



実行中に変数の値を変更できる

デバッガーの自作

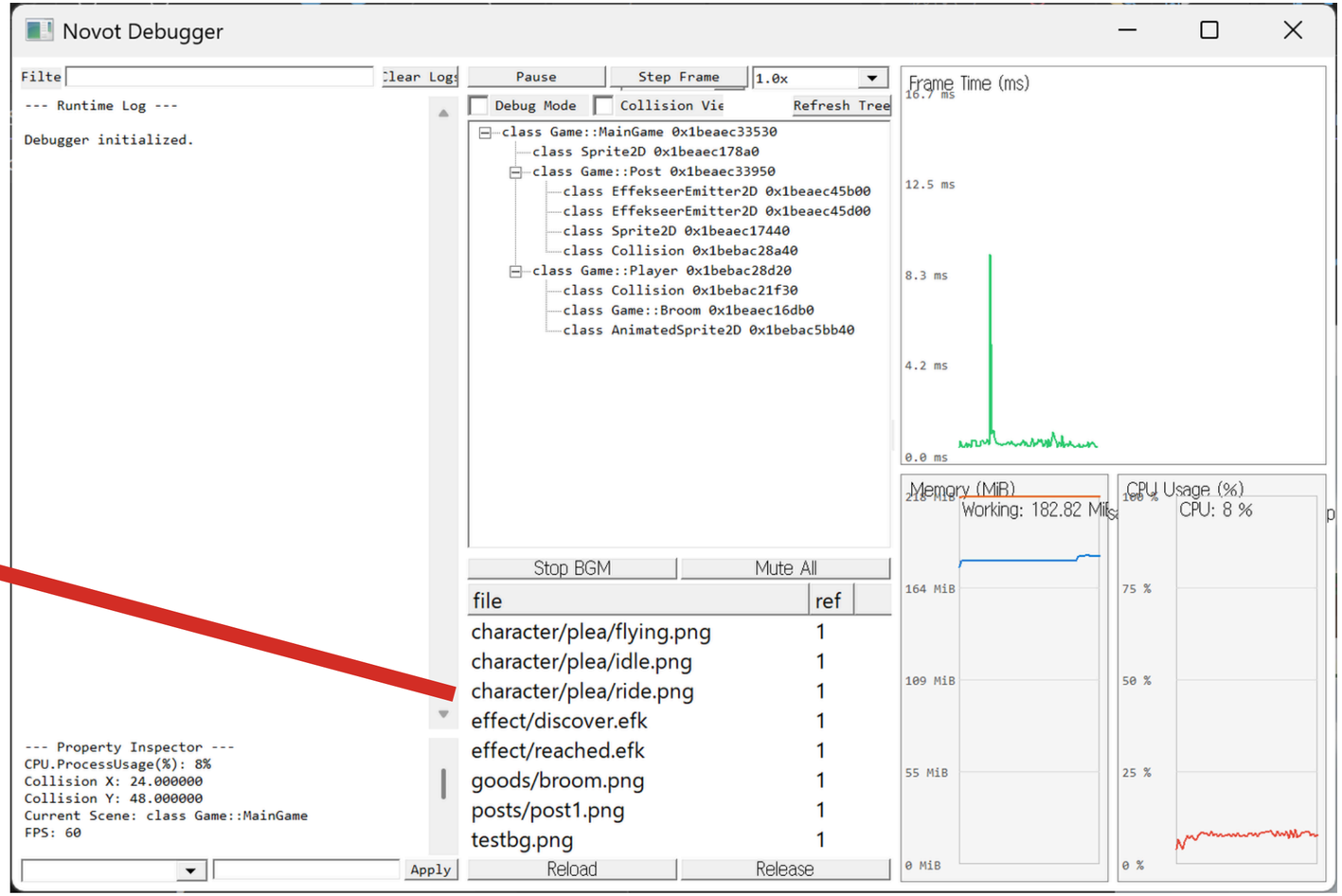
シーンツリー



今のシーン構造の把握

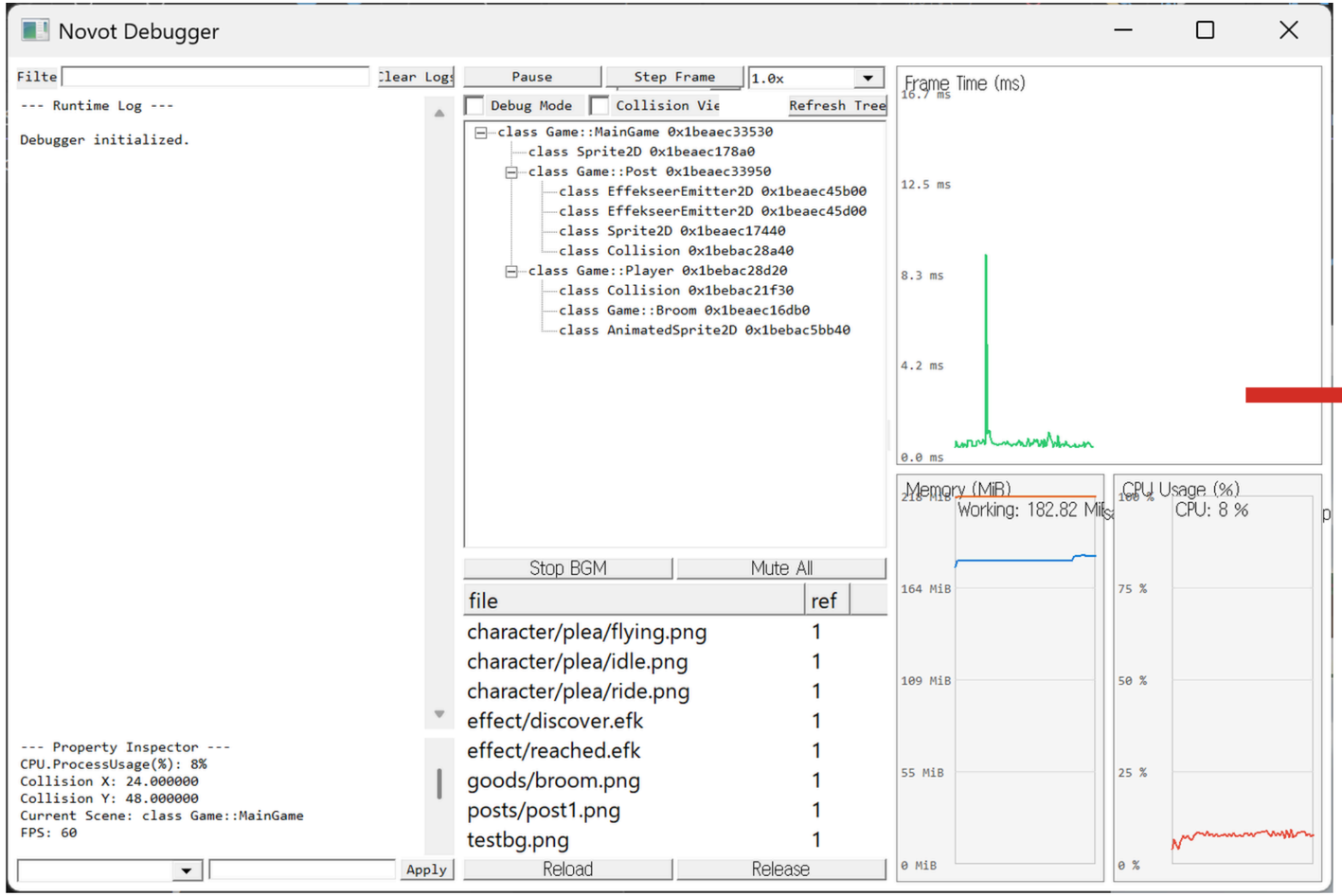
デバッガーの自作

アセットツリー



アセットの管理

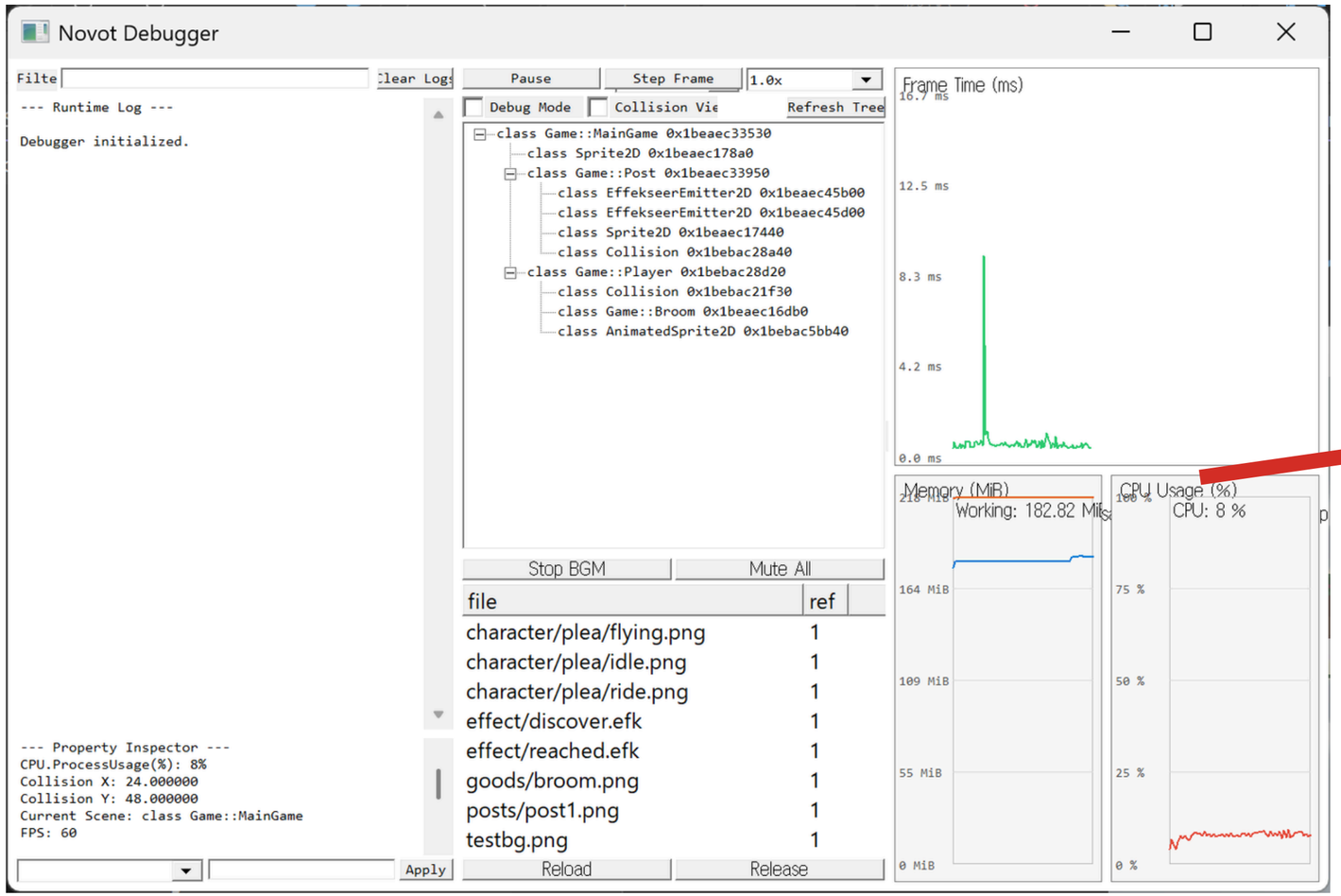
デバッガーの自作



処理時間グラフ

かっこいい(フレームスパイク監視)

デバッガーの自作



メモリ&CPU

かっこいい(パフォーマンス監視)

デバッガーの自作

関連フレームワーク: imgui
(任天堂もマリカで使ってるらしい)

アセットの管理

アセットの管理



↑同じ画像を三回もロード…？

アセットの管理

参照カウンタを導入してみる(shared_ptrみたいな)

- シーン単位で使用できる
- まだ読み込んでなければ読み込み、ref=1
- 0 < ref なら代わりに ref++ し、キャッシュを使用
- シーン解放時に使用中のアセットを使用分ref--
- シーン解放時のデクリメントでref <= 0なら解放

とりあえずここまで！！！！！


{ the first volume }
Game Engine Architecture
第 4 版
ゲームエンジンアーキテクチャ
上
著者 ジェイソン・グレゴリー
監修 今給黎 隆／湊 和久
ブラックボックスを開けた人から、強くなっていく。
生成AIでゲームが作れる時代だからこそ身体に染み込ませたい基礎力がここに。
監修 今給黎 隆
C++23とメモリの基礎からロックフリー・SIMD・GPGPUの並行処理

これ読む人募集中

尾張